SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Dry Run Evaluation

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO1 – Apply (BL3)	**Assessment:**	Assessment Tool 2 – Dry Run Evaluation
Weightage:	35% of CIA	**Duration / Marks:**	60 mins / 100 marks
CO1 Statement:	Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)		
Student Name:	**Afrose A**	**Reg. No.:**	**1924260309**
Section / Batch:		**Date:**	

Code of Conduct

I **Afrose A**, **1924260309** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

Instructions

- Read each question carefully; answer all four questions (25 marks each, 100 in total).
- Show every dry-run step: the initial queue, stack, priority queue or belief state, each iteration, your assumptions and a clear final answer, and finish with the interpretation asked for.
- Follow the conventions stated in each question (alphabetical expansion, explored set, goal test on removal from the frontier, tie-breaking) and present tables neatly.
- Marks follow the Dry Run Evaluation rubric: understanding of the search problem, state expansion and dry run execution, application of the search algorithm, accuracy of the solution path, and presentation and logical reasoning.
- Upload in GitHub and GCR (question paper with Reg. No., Name, answers, rubrics and evaluation page at the end). Duration: 1 hour (15 minutes per question).

Section / Topic	Focus	Question Nos.
1. Foundations and History of AI	Four approaches to AI, Turing Test, eras and milestones	**Q2**
2. Intelligent Agents: Agents and Environments	Agent, percepts, actions, agent function and program, PEAS	**Q1**
3. The Concept of Rationality	Performance measure, percept sequence, rational vs omniscient	**Q1**
4. The Nature of Environments	Task-environment properties and their classification	**Q1**
5. The Structure of Agents	Simple reflex and model-based reflex agent programs	**Q1**
6. Problem-Solving Agents	Goal formulation, problem formulation, search, execution	**Q2**
7. Problem Formulation	States, initial state, actions, goal test, path cost; belief states	**Q2, Q4**
8. Searching for Solutions	Search graph, frontier, explored set, node expansion	**Q3**

9. Uninformed Search Strategies	Dry runs of BFS, DFS and uniform-cost search	**Q3**
10. Search with Partial Information	Sensorless problems, belief states, percept update	**Q4**

Annexure A – Dry Run Evaluation

Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

Topic: Intelligent agents, rationality, nature of environments, structure of agents | 15 minutes

A vacuum-cleaner agent lives in a world with two squares, A and B. At each time step it perceives [its location, status of its own square] and performs one action: Suck, Right, Left or NoOp. The agent starts in square A, both squares are initially Dirty, and dirt never reappears.

Performance measure over T = 6 time steps: +10 for every square cleaned (a Suck on a Dirty square) and −1 for every Left or Right move.

Agent X (simple reflex): if the current square is Dirty, Suck; otherwise move to the other square.

Agent Y (model-based reflex): keeps an internal state of the squares it knows to be clean. If the current square is Dirty, Suck; otherwise, if both squares are known clean, NoOp; otherwise move to the other square.

(a) Write the PEAS description (Performance measure, Environment, Actuators, Sensors) of this agent. **[4 Marks]**

(b) Classify the task environment on observability, number of agents, determinism, episodic/sequential, static/dynamic and discrete/continuous, with a one-phrase justification each. **[4 Marks]**

(c) Dry-run both agents for all 6 time steps. Complete the table (percept, action, and Y's internal state after the step). **[9 Marks]**

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares
1	[A, Dirty]	Suck	[A, Dirty]	Suck	{A}
2	[A, Clean]	Right	[A, Clean]	Right	{A}
3	[B, Dirty]	Suck	[B, Dirty]	Suck	{A, B}
4	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}
5	[A, Clean]	Right	[B, Clean]	NoOp	{A, B}
6	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}

(d) Compute the total performance of X and of Y over the 6 steps. **[3 Marks]**

(e) Interpret: which agent is rational? Both agents receive the identical percept [B, Clean] at step 4 – explain why they act differently, using the structure of the two agent programs, and distinguish a rational agent from an omniscient agent. **[5 Marks]**

ANSWERS

(a) PEAS - Performance: +10 per square cleaned, -1 per Left/Right move (T=6). Environment: two squares A and B, each Clean/Dirty, dirt never reappears. Actuators: Suck, Left, Right, NoOp. Sensors: location sensor + dirt sensor for the current square.

(b) Partially observable (senses only its own square); single agent (no other agent); deterministic (actions always have the intended effect); sequential (current action affects future reward); static (world does not change while agent decides); discrete (finite squares, percepts, actions).

(d) Table above. X: cleans 2 squares (+20), makes 4 moves at steps 2, 4, 5, 6 (-4), total = 16. Y: cleans 2 squares (+20), makes 1 move at step 2 (-1), total = 19.

(e) Agent Y is rational: it maximises expected performance given its percept sequence and knowledge - it stops moving once both squares are known clean. X keeps shuttling between clean squares and loses 1 point on every move, so it is not rational here. At step 4 both get the same percept [B, Clean]. X is a simple reflex agent: its program maps the current percept only ("not Dirty -> move to the other square"), so it goes Left. Y is a model-based reflex agent: its internal state records {A, B} as known clean, so its rule "both known clean -> NoOp" fires. Same percept, different internal state, different action. A rational agent maximises EXPECTED performance using what it has perceived; an omniscient agent knows the ACTUAL outcome of every action in advance, which is impossible in practice. Rationality is not omniscience or perfection.

Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

Topic: Foundations and history of AI, problem-solving agents, problem formulation | 15 minutes

Part A. Complete the table by classifying each system or idea under one of the four approaches to AI (thinking humanly, acting humanly, thinking rationally, acting rationally) and naming the era or milestone of AI history to which it belongs.

(a) Complete the table. **[8 Marks]**

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed (1950)	Acting humanly	Gestation of AI (1943-1955); Turing's 1950 paper 'Computing Machinery and Intelligence'
General Problem Solver (Newell and Simon, 1957)	Thinking humanly (cognitive modelling)	Birth of AI (Dartmouth 1956) / early enthusiasm and great expectations (1952-1969)
Aristotle's syllogisms and formal logic	Thinking rationally (laws of thought)	Philosophical foundations - logic, c. 350 BC (Aristotle)
An agent that does the right thing to maximise expected performance	Acting rationally (rational agent approach)	Emergence of intelligent agents (1995 - present)

Part B. A problem-solving agent has two jugs: X of capacity 4 litres and Y of capacity 3 litres, both initially empty and without measuring marks. It may fill a jug from a tap, empty a jug on the ground, or pour one jug into the other until the source is empty or the target is full. The goal is to have exactly 2 litres in jug X.

(b) Formulate the problem: state representation, initial state, actions, transition model, goal test and path cost. **[6 Marks]**

(c) Generate all successor states of (0, 0) and of (4, 0) (leave out actions that leave the state unchanged). **[3 Marks]**

(d) How many states does the state space contain? Write a shortest solution as a sequence of states and give its path cost. **[3 Marks]**

(e) Interpret: describe the four phases (goal formulation, problem formulation, search, execution) of a problem-solving agent for this task, and explain why the agent can execute the found solution without looking at new percepts. **[5 Marks]**

ANSWERS

(b) State = (x, y), x = litres in X (0..4), y = litres in Y (0..3). Initial state = (0, 0). Actions: Fill X, Fill Y, Empty X, Empty Y, Pour X->Y, Pour Y->X. Transition: Fill X -> (4, y); Fill Y -> (x, 3); Empty X -> (0, y); Empty Y -> (x, 0); Pour X->Y: t = min(x, 3-y) gives (x-t, y+t); Pour Y->X: t = min(y, 4-x) gives (x+t, y-t). Goal test: x = 2. Path cost: 1 per action (step cost 1).

(c) Successors of (0,0): (4,0) by Fill X; (0,3) by Fill Y. Successors of (4,0): (4,3) by Fill Y; (0,0) by Empty X; (1,3) by Pour X->Y.

(d) Total possible (x, y) combinations = 5 x 4 = 20 states (14 are reachable from (0,0)). Shortest solution: (0,0) -> (0,3) -> (3,0) -> (3,3) -> (4,2) -> (0,2) -> (2,0). Path cost = 6 actions.

(e) 1) Goal formulation: the goal is the set of states with exactly 2 L in jug X. 2) Problem formulation: decide the states, actions, transition model, goal test and cost described in (b). 3) Search: find an action sequence leading from (0,0) to a goal state (e.g. BFS gives the 6-step plan: Fill Y, Pour Y->X, Fill Y, Pour Y->X, Empty X, Pour Y->X). 4) Execution: carry out the actions one by one. The agent can execute the plan without looking at new percepts because the environment is fully observable at planning time, known, deterministic and static - every action has exactly the predicted result, so the plan is executed open-loop and the agent never needs to check the outcome.

Question 3: Searching for Solutions – Uninformed Search Strategies

Topic: Searching for solutions, BFS, DFS, uniform-cost search | 15 minutes

Consider the weighted undirected graph below. The start node is S and the goal node is G. Use graph search with an explored set; expand neighbours alphabetically, test the goal when a node leaves the frontier, and break ties alphabetically. BFS and DFS ignore edge costs while searching, but path costs are measured with them.

Edge	Cost	Edge	Cost
S – A	1	B – D	1
S – B	4	C – G	6
A – C	2	D – G	2
A – D	5		

(a) Perform BFS from S (queue initially [S]). Do not add a node that is already explored or already in the queue. **[6 Marks]**

Iteration	Node Dequeued	Queue after Expansion	Explored Set
1	S	[A, B]	{S}
2	A	[B, C, D]	{S, A}
3	B	[C, D]	{S, A, B}
4	C	[D, G]	{S, A, B, C}
5	D	[G]	{S, A, B, C, D}
6	G (goal reached)	[ ]	{S, A, B, C, D}

(b) Perform DFS from S using a stack (initially [S]); push children so that the alphabetically first child is popped first. Write the stack with its top on the left. **[5 Marks]**

Iteration	Node Popped	Stack after Pushing Children
1	S	[A, B]
2	A	[C, D, B]
3	C	[G, D, B]
4	G (goal reached)	[D, B]

(c) Perform uniform-cost search from S. Write queue entries as Node:g(parent) and replace an entry whenever a cheaper path to a frontier node is found. **[7 Marks]**

Iteration	Node Expanded	g(n)	Priority Queue after Expansion
1	S	0	[A:1(S), B:4(S)]
2	A	1	[C:3(A), B:4(S), D:6(A)]
3	C	3	[B:4(S), D:6(A), G:9(C)]
4	B	4	[D:5(B), G:9(C)] D replaced 6->5
5	D	5	[G:7(D)] G replaced 9->7
6	G (goal reached)	7	[ ]

(d) Complete the comparison table. **[4 Marks]**

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS	S - A - C - G	9	6 (S, A, B, C, D, G)	No
DFS	S - A - C - G	9	4 (S, A, C, G)	No

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
UCS	S - B - D - G	7	6 (S, A, C, B, D, G)	Yes

(e) Interpret: explain why UCS removed as many nodes as BFS yet found a cheaper path, why DFS needed the fewest nodes here but cannot be trusted in general, and why an explored set is needed in graph search (use iteration 3 of BFS as an example). **[3 Marks]**

ANSWER (e)

1. UCS removed 6 nodes, the same as BFS (S, A, C, B, D, G vs S, A, B, C, D, G), but it orders the frontier by path cost g(n) instead of by depth. BFS only minimises the number of edges, so it returned S-A-C-G (3 edges, cost 9); UCS discovered that going S-B-D-G (cost 4+1+2 = 7) is cheaper, replacing D:6 by D:5 and G:9 by G:7, so it found the cheaper optimal path.
2. DFS needed the fewest nodes (4) because it dived down the first branch S-A-C and happened to hit G. DFS is not optimal and not guaranteed to be complete - it ignores costs, returns the first path it finds (cost 9 here, not 7), and may follow very deep or infinite branches. So it cannot be trusted in general.
3. The explored set stops the search from re-expanding nodes and looping in an undirected graph (every edge can be walked back). At BFS iteration 3, B is dequeued; its neighbours are S (already explored) and D (already in the queue), so nothing is added. Without the explored set/queue check, S and D would be added again as duplicates, causing repeated work and possibly infinite loops.

Question 4: Search with Partial Information – Belief States

Topic: Problem formulation, search with partial information (sensorless and percept-based belief-state search) | 15 minutes

A vacuum agent lives in a two-square world (Left square, Right square), each square being Clean (C) or Dirty (D). Actions: Left, Right and Suck (Left in the left square and Right in the right square leave the agent where it is; Suck cleans the current square). Physical states are written (Agent, Left square, Right square):

State	Description	State	Description
S1	(L, D, D)	S5	(L, C, D)
S2	(R, D, D)	S6	(R, C, D)
S3	(L, D, C)	S7	(L, C, C)
S4	(R, D, C)	S8	(R, C, C)

(a) Initially the agent has no sensors and does not know its location or the dirt status. Formulate the problem over belief states: initial belief state, actions, goal test and path cost. **[3 Marks]**

(b) Apply the action sequence [Right, Suck, Left, Suck] and write the belief state after each action. **[6 Marks]**

Step	Action	Belief State after Action
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	{S2, S4, S6, S8}
2	Suck	{S4, S8}
3	Left	{S3, S7}
4	Suck	{S7} (goal reached)

(c) Now the agent has a local sensor that reports [location, status of the current square]. Starting again from the initial belief state, the first percept is [Left, Dirty]; the agent then performs Suck and perceives [Left, Clean]; it then performs Right and perceives [Right, Dirty]. Fill in the belief state after each action (prediction) and after each percept (update). **[8 Marks]**

Step	Action	After Action (Predicted)	Percept	After Percept (Updated)
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}	[Left, Dirty]	{S1, S3}
1	Suck	{S5, S7}	[Left, Clean]	{S5, S7}
2	Right	{S6, S8}	[Right, Dirty]	{S6}

(d) How many different belief states are possible when there are 8 physical states? Why does this make the problem harder than searching over physical states? **[3 Marks]**

(e) Interpret: explain why the percepts in part (c) shrink the belief state, how this differs from the sensorless case in part (b), and why the sensorless plan can still be called rational. **[5 Marks]**

ANSWERS

(a) Initial belief state = {S1,...,S8} (agent could be in any state). Actions: Left, Right, Suck. Goal test: every state in the belief state is a goal, i.e. belief state is a subset of {S7, S8}. Path cost: 1 per action.

(b),(c) Table (b) and (c) are filled above. Belief states after the sensorless plan: Right -> {S2,S4,S6,S8}; Suck -> {S4,S8}; Left -> {S3,S7}; Suck -> {S7}.

(d) A belief state is any subset of the 8 physical states, so there are 2^8 = 256 belief states (255 if the empty set is excluded). Search over belief states is exponentially larger than over the 8 physical states, and each action/percept must update a whole set of states.

(e) A percept removes every state inconsistent with the observation: [Left, Dirty] keeps only the states with the agent in the left square and that square dirty, {S1, S3}; later [Right, Dirty] shrinks {S6, S8} to {S6}. In the sensorless case in (b) there are no percepts; the belief state shrinks only because actions merge states (e.g. Right, Suck, Left, Suck coerces all 8 states into S7). The sensorless plan is still rational because it is guaranteed to reach the goal from every possible initial state, so it maximises expected performance given the agent's knowledge.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Accuracy of Solution Path	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Q. No. / Criteria	Understanding of Search Problem	State Expansion and Dry Run Execution	Application of Search Algorithm	Accuracy of Solution Path	Accuracy of Solution Path	Sub. Total (20)	Total %
1							
2							
3							
4							
Total							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____